

FluidPD: In-Place Elasticity for SLO-Aware Prefill-Decode Disaggregated LLM Serving

Kartik Ramesh¹, Kaidi Fu¹, Zihan Zheng¹, Jiahuan Yu¹, Fabio Oliveira², Carlos Costa², Minjia Zhang¹

¹University of Illinois Urbana-Champaign

²IBM Research

Abstract—Prefill-decode disaggregation is becoming a common architecture for LLM serving because it separates two phases with distinct execution patterns and SLO objectives. Existing systems typically combine a fixed prefill/decode worker ratio with request routing across workers. However, real-world workloads exhibit both short bursts and sustained shifts in the prefill-to-decode demand ratio. As a result, a configuration that is well provisioned at one time may quickly become mismatched, causing latency SLO violations even when idle capacity exists elsewhere. Existing autoscaling mechanisms can add capacity, but they react slowly, require spare GPUs, and do not directly address short-timescale phase imbalance.

We present FluidPD, a P/D-disaggregated serving system that provides SLO-aware in-place elasticity. FluidPD introduces two complementary mechanisms. FluidToken handles transient imbalance by offloading a bounded portion of prefill computation to decode workers when decode-side slack is available. FluidRole handles sustained imbalance by reassigning running workers between prefill and decode roles in place, avoiding model reload and engine restart. Both mechanisms are guided by lightweight pressure indices that expose prefill and decode-side resource pressure before they appear as SLO violations. Across production Azure trace workloads, FluidPD improves overall SLO attainment over static SGLang by up to 94.6 percentage points, demonstrating that SLO-aware in-place P/D elasticity improves service quality without provisioning additional workers.

I. INTRODUCTION

Large language models (LLMs) are increasingly deployed in interactive applications, such as chat assistant [32], coding agent [5], [14], [33], and tool-using workflows [30], [34], [50]. These applications have strict latency demands on both initial response time and continuous token generation. Prefill-decode (P/D) disaggregation has become a common architecture for meeting these demands in LLM serving [35], [54]. By placing prefill and decode on separate worker pools, it allows each phase to be provisioned and optimized around its own execution pattern and Service Level Objectives (SLOs). Prefill processes the input prompt and largely determines Time-To-First-Token (TTFT), whereas decode generates output tokens iteratively and determines Time-Per-Output-Token (TPOT). This separation makes P/D disaggregation effective for high-throughput, SLO-aware LLM serving.

However, this separation also makes capacity difficult to move across phases. In a typical deployment, the numbers of prefill and decode workers are chosen ahead of time, often through offline workload profiling [35], [54], and each worker remains tied to its assigned role at runtime. This

static design has limited flexibility under dynamic workloads. Production LLM workloads exhibit both short-term bursts and sustained shifts in request mix [42]. A burst can temporarily overload prefill workers when many prompts arrive within a short interval [47], whereas a sustained shift can change the aggregate prefill-to-decode demand ratio as the workload mix moves, for example, between prompt-heavy coding requests and longer interactive conversations [54]. Under a static P/D partition, routing can balance requests only among workers already assigned to the required phase [38], [54]. It cannot use idle decode-side capacity to relieve prefill pressure, nor can it reassign underutilized prefill capacity when decode demand grows. Consequently, the system may violate latency SLOs even when sufficient capacity remains available elsewhere in the running deployment.

Autoscaling is a natural response to changing demand and is increasingly supported by production LLM serving platforms [1], [9], [27]. It is effective when the running deployment lacks sufficient aggregate capacity, but is less suitable when capacity is merely distributed poorly between prefill and decode. Scale-out requires spare GPUs and incurs detection, provisioning, model loading, and runtime initialization delays. These delays are too long for short bursts, while scaling out for sustained phase imbalance adds resources even when the less-pressured pool already has available capacity. In our Dynamo [9] experiment, a new worker becomes ready ~ 200 s after scale-out is triggered, long after the burst has ended. We call this missing capability *in-place elasticity*: adapting the prefill/decode capacity available within the running deployment before changing its serving world size.

We present FluidPD, the first P/D-disaggregated serving system to combine SLO-aware token-level work stealing for transient pressure with bidirectional in-place role switching for sustained phase imbalance. FluidPD adapts the prefill and decode capacity of a fixed set of running workers without changing the serving world size. For short-lived prefill pressure, FluidToken offloads a bounded suffix of selected prefill requests to their paired decode workers, allowing available decode-side capacity to reduce the prefill backlog without changing worker roles. When phase pressure persists, FluidRole performs lightweight, bidirectional role switching between prefill and decode, rebalancing the P/D worker split within the running deployment without dropping in-flight requests or provisioning additional workers. Both mechanisms

are guided by two predictive pressure signals derived from internal engine state. The Prefill Pressure Index (PPI) estimates whether queued prefill requests can meet their remaining TTFT budgets, while the Decode Pressure Index (DPI) captures decode execution pressure and KV-cache admission headroom. Unlike observed TTFT and TPOT, which reveal pressure only after it has affected request latency, these signals expose emerging prefill and decode pressure early enough to guide adaptation. Together, FluidPD is able to respond to both bursts and sustained workload shifts using capacity already available in the deployment. In summary, this paper makes the following contributions:

- We show that static P/D partitions can lose SLO attainment under both short bursts and sustained demand shifts, even when aggregate capacity remains sufficient. Autoscaling is too slow for transient pressure and too coarse for phase imbalance.
- We present FluidPD, the first P/D-disaggregated serving system to combine SLO-aware token-level work stealing with lightweight, bidirectional in-place role switching. Its decisions are guided by predictive PPI and DPI signals derived from internal engine state, allowing it to respond to transient and persistent pressure before latency violations happen.
- We implement FluidPD on SGLang and evaluate it on production-derived Azure workloads across three LLMs. FluidPD improves SLO attainment over SGLang by up to 94.6 percentage points under the same GPU allocation, and outperforms Dynamo autoscaling by up to 88.3 percentage points while using fewer GPUs.

II. RELATED WORK

A large body of work improves LLM serving efficiency through advanced batching and scheduling [3], [11], [52], KV-cache and memory management [13], [20], [23], [53], optimized attention and decoding kernels [4], [10], [31], [37], [46], [51], CPU offloading [40], [49], model parallelism and pipelining frameworks [24], [36], and speculative decoding [28], [41]. Other systems use request ordering, admission control, and batch selection to meet latency SLOs [7], [8], [15], [19], [44]. These techniques improve the efficiency and scheduling of co-located prefill–decode execution. FluidPD instead targets P/D-disaggregation, where prefill and decode run on separate worker pools.

To better accommodate the different compute characteristics of the prefill and decode phases, recent work has proposed P/D-disaggregation, which assigns the two phases to separate GPU workers or worker pools. Systems such as Splitwise [35], TetriInfer [16], Llumnix [43], and DistServe [54] show that this separation can improve goodput and TTFT and ITL SLO attainment while enabling phase-specific execution policies. Subsequent work introduces additional optimizations for provisioning, scheduling, and KV-cache management [16], [38]. Industry studies further show that the preferred P:D partition depends on the workload and can change over time [22], [29].

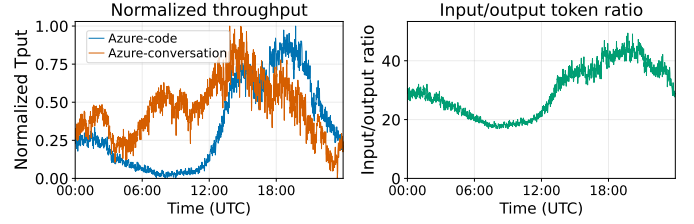


Fig. 1: Time-varying input/output ratios in Azure workloads.

However, these designs primarily optimize a given P/D configuration and do not continuously adapt the running capacity split as phase demand changes.

Several systems adapt phase capacity by changing worker roles or serving groups. Arrow [48] routes requests over stateless instances and reallocates instances between prefill and decode. DOPD [26] adjusts instance allocation toward a predicted target P/D ratio, while P/D-Serve [18] supports dynamic P/D organization at deployment scale. ThunderServe [17] uses lightweight rescheduling to change phase assignments and request orchestration in heterogeneous cloud environments. At a broader deployment level, NVIDIA Dynamo [9], AIBrix [1], and llm-d [27] provide autoscaling, routing, KV-cache management, and orchestration for distributed inference. These systems adapt capacity at worker or serving-group granularity, but do not exploit token-level work stealing for short-lived phase pressure.

Other systems share capacity across the P/D boundary without first reconfiguring whole workers. TaiChi [45] and WindServe [12] support mixed prefill–decode execution within an instance, while TokenScale [21] uses pre-provisioned convertible decode workers to absorb prefill bursts. Libra [39], Kairos [6], and PPD [25] further move request execution across phase boundaries at token, request, or conversational-turn granularity. These systems provide either mixed execution or specialized cross-phase offloading, but do not combine SLO-bounded token-level work stealing for transient pressure with in-place worker-role adaptation for sustained changes in the P/D demand ratio.

III. OBSERVATIONS AND OPPORTUNITIES

P/D-disaggregated serving uses separate prefill and decode worker pools, but a fixed capacity split may not track changing workloads. We show that short bursts and sustained demand shifts both create phase imbalance.

A. Sustained Demand Shifts Break SLOs under Static P/D Splits

Different LLM applications produce substantially different prompt and generation characteristics. Prompt-heavy workloads generally consume relatively more prefill capacity, whereas generation-heavy workloads place greater relative demand on decode. Consequently, changes in the application mix can shift the effective P/D capacity split required by the workload over time. Fig. 1 shows the diurnal variation using one day from the Azure trace [35]. We observe that

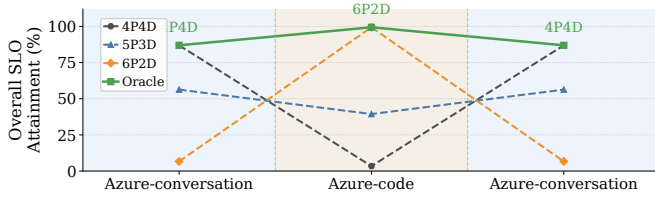


Fig. 2: Sustained shifts favor different static P/D ratios.

coding requests are concentrated during business hours and tend to be more prefill-heavy, while conversational requests are distributed more evenly throughout the day and contain relatively more decode work. As the workload mix changes over the day, the aggregate input-to-output token ratio varies substantially, from 16.9 to 49.4.

A static P/D deployment cannot adapt to this shift because worker capacity is bound to phase-specific roles. As demand moves between prefill and decode, a fixed P/D partition can become poorly matched to the workload even when aggregate capacity is sufficient. To measure the SLO impact of this workload shift, we construct *Azure (staged)*, a controlled workload with three consecutive 10k-request windows that follow an Azure-conversation, Azure-code, Azure-conversation sequence; its construction is described in detail in § V-A. We evaluate the workload using Llama-3.1-8B at a constant 40 RPS Poisson arrival rate on an 8×A100-80GB server and compare 4P4D, 5P3D, and 6P2D deployments, using TTFT and TPOT SLOs of 2000ms and 100ms. We define the oracle P/D ratio for each window as the static ratio with the highest overall SLO attainment. Fig. 2 shows that no single static partition performs well across both workload types: 4P4D reaches 86.93% SLO attainment on Azure-conversation but only 3.40% on Azure-code. In contrast, 6P2D reaches 99.40% on Azure-code but only 6.64% on Azure-conversation. The oracle performs well by selecting a different P/D split for each window, which shows that the SLO loss comes from P/D imbalance rather than insufficient total capacity.

B. Short Bursts Create Transient P/D Imbalance

Workload variation also occurs at a much shorter timescale. We define a burst as a brief interval in which arrival or token demand rises sharply. Unlike the sustained shifts in § III-A, a burst does not necessarily change the long-term workload mix. To measure the effect of such bursts, we construct *Azure (mixed)*, a 10K-request workload composed of Azure-code and Azure-conversation requests; its construction is described in detail in § V-A. We replay it using Llama-3.1-8B on 8×A100-80GB under a static 4P4D deployment, with TTFT and TPOT SLOs of 2000 ms and 100 ms, respectively. Fig. 3 shows the resulting burst-induced prefill pressure. During the marked burst, input-token demand rises from a median of 98.0K to 109.3K tokens/s. The burst rapidly builds a prefill waiting queue, causing TTFT to exceed its SLO. More importantly, the impact of this burst lasts longer than the burst itself. New requests arriving during and after the burst must wait behind

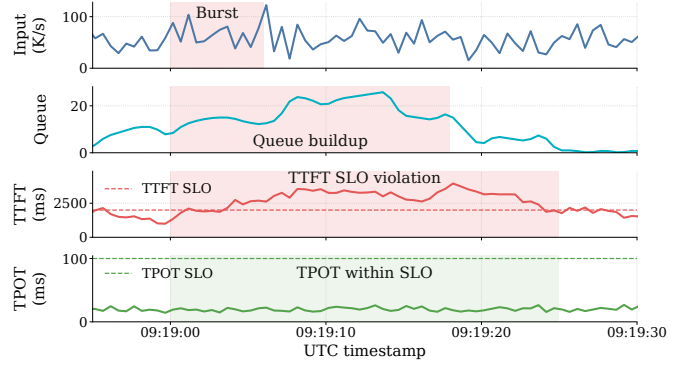


Fig. 3: Transient bursts build prefill queueing, causing TTFT violations while TPOT remains within SLO.

the accumulated prefill backlog, so TTFT remains high even after input-token demand begins to fall. Meanwhile, TPOT remains within its SLO throughout the interval. Requests enter decode only after completing prefill, and their output tokens are produced iteratively rather than arriving as one prompt-sized unit of computation. This immediate burst pressure therefore accumulates primarily in the prefill queue, while already admitted decode streams continue to make progress within their per-token latency budget. This behavior exposes another limitation of static P/D serving: a short burst can overload the prefill pool while decode workers still have slack, yet phase-specialized workers cannot use that capacity to reduce the prefill backlog. Moreover, the pressure may disappear before changing the cluster-level P/D split becomes worthwhile, which motivates a fine-grained adaptation method that temporarily exploits decode-side slack without changing worker roles.

C. Autoscaling Is Often Coarse for P/D Imbalance

Autoscaling is a common approach to handling changes in inference demand. Systems such as Dynamo, AIBrix, and llm-d can add or remove inference workers as load changes [1], [9], [27]. This capability is important when total workload demand changes over long timescales, but it is less suitable for the two forms of phase imbalance identified above.

For short bursts, autoscaling incurs both detection and actuation delay. The controller must first observe a lagging symptom, such as latency degradation or high resource utilization. By the time the scale-out condition is detected, the burst may already have produced a substantial backlog. After the decision, the newly allocated capacity must be provisioned before it can serve requests.

To quantify these delays, we deploy Llama-3.1-8B with Dynamo’s SGLang backend and disaggregated planner on the same Azure (mixed) workload. Starting from a 3P3D deployment under an 8-GPU budget, Dynamo issues a scale-out decision roughly 5 s after the burst begins. However, the added worker becomes ready only about 200 s later: approximately 100 s elapses before Kubernetes creates the pod, including scheduling and resource provisioning, and a further

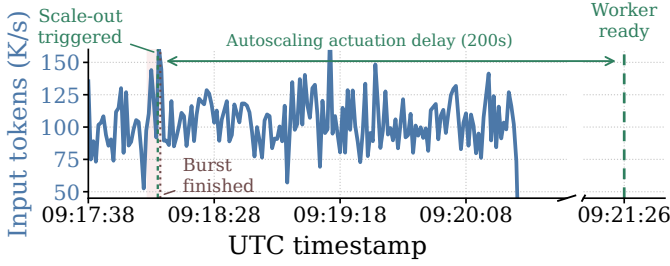


Fig. 4: Dynamo triggers scale-out during the burst, but the new worker is ready only after it ends.

~100 s is spent initializing the inference runtime, loading the model weights, and capturing CUDA graphs before the worker can serve requests. The burst, and in fact even the synthetic workload, has long ended before the added capacity becomes available, making autoscaling unable to mitigate the resulting prefill backlog and TTFT violations.

Autoscaling is also unnecessarily expensive when the problem is a phase-capacity imbalance rather than insufficient aggregate capacity. One worker pool may be overloaded while the other has substantial slack. Adding another worker increases the serving world size and consumes additional GPUs, even though sufficient resources may already exist inside the running deployment. Autoscaling is therefore well suited to aggregate-capacity shortages, but can be too slow for short bursts and too coarse when capacity is imbalanced across prefill and decode.

IV. SYSTEM DESIGN

FluidPD provides in-place elasticity for P/D-disaggregated LLM serving. Its goal is to adapt the effective prefill/decode capacity available to a running deployment without immediately changing the serving world size.

A. Overview

Fig. 5 shows the architecture of FluidPD. FluidPD builds on the standard P/D-disaggregated structure with separate prefill and decode worker pools and a router that assigns each request to a prefill/decode worker pair. FluidPD then introduces two elasticity mechanisms. FluidToken provides request-level adaptation. When a prefill worker estimates that its queued requests are at risk of violating TTFT, it can ask the paired decode worker to execute a suffix of selected prefill requests. This token-level work stealing reduces prefill queuing delay without changing worker roles. Since the stolen work runs on a decode worker, FluidToken allows stealing only when the decode worker has sufficient slack and memory headroom, preventing TTFT gains from causing TPOT violations or decode-side backpressure.

FluidRole provides worker-level adaptation. When phase pressure persists, FluidPD changes the effective P/D ratio by switching selected workers between prefill and decode roles in place. Unlike auto-scaling, FluidRole keeps model weights and runtime state, and changes the active role of an existing

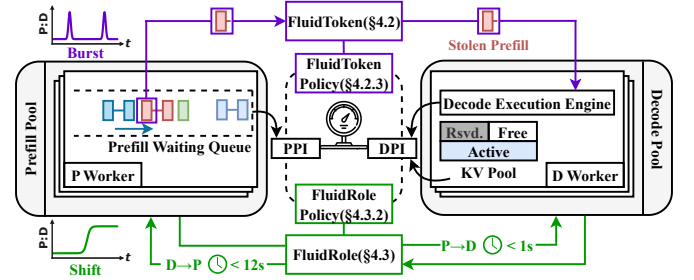


Fig. 5: Overview of FluidPD. FluidToken absorbs transient prefill bursts while FluidRole handles sustained P/D demand shifts.

worker after draining in-flight work. The two mechanisms are coordinated by pressure-guided control, i.e., Prefill Pressure Index (PPI) and Decode Pressure Index (DPI). FluidToken is used first because it can relieve transient prefill pressure without changing worker roles. FluidRole is invoked when pressure persists, indicating that the current P/D partition is no longer suitable for the workload. Together, they allow FluidPD to handle both short bursts and sustained P/D demand shifts within the running deployment, while reserving autoscaling for cases where the workload exceeds the available cluster capacity.

B. FluidToken: Pressure-Guided SLO-Aware Token Stealing

1) *Token-Stealing Execution*: FluidToken steals work from prefill to decode, targeting the short-term bursts in § III-B, where prefill queues build while decode-side slack remains available. It implements this stealing as a partial-prefill handoff along the request’s existing P/D execution flow, as shown in Fig. 6. For a request i with prefill length L_i , FluidToken selects a *stealing ratio* $\beta_i \in [0, 1]$ using the policy in § IV-B3, yielding a prefill-side prefix of length $L_{\text{pfx}} = (1 - \beta_i)L_i$ and a decode-side suffix of length $L_{\text{sfx}} = \beta_i L_i$.

After the prefill worker computes the prefix and transfers its KV states, the decode worker executes the suffix by treating the transferred KV as prior context, as in chunked prefill [3]. Each suffix token therefore observes the same causal mask and position ordering as it would in an unsplit prefill, making split execution mathematically equivalent to unsplit. After prefilling the suffix, the decode worker generates the first token and inserts the request into the decode batch.

As shown in Fig. 6, FluidToken reuses SGLang’s async P/D KV-transfer path [53]. During the prefix-KV transfer, both workers continue processing other requests, overlapping the handoff with concurrent prefill and decode execution. FluidToken therefore redistributes rather than reduces prompt computation. Its benefit comes from moving part of the queued prefill work to a decode worker with available slack, allowing the prefill worker to begin subsequent requests earlier. Fig. 7 illustrates this effect. FluidToken splits request R_2 , and schedules just the prefix $R_{2\text{pfx}}$ on the prefill worker. This allows request R_3 to start prefill earlier and meet its TTFT SLO.

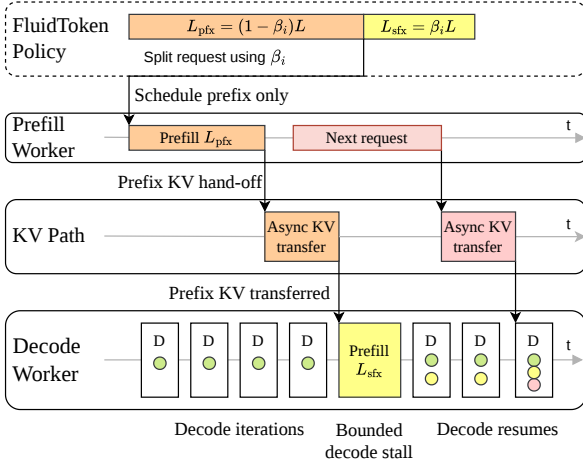


Fig. 6: FluidToken execution flow for prefix KV handoff and stolen-suffix execution.

Executing a stolen suffix introduces two decode-side costs. First, the suffix forward pass temporarily defers the next decode iteration. Second, earlier admission of the stolen request increases decode-side KV occupancy and may reduce the worker’s headroom for subsequent requests.

To address this, FluidToken admits stolen work only when the paired decode worker has sufficient TPOT slack and KV-cache headroom. The pressure signals and policy that enforce these conditions and determine the maximum admissible suffix length are described next.

2) *Prefill-Decode Pressure Signals*: FluidToken uses two phase-specific signals. The Prefill Pressure Index (PPI) predicts whether queued requests are at risk of missing their TTFT SLOs and therefore whether token stealing is needed. The Decode Pressure Index (DPI) captures whether the paired decode worker has sufficient resource and admission headroom to accept stolen work.

Directly using observed TTFT and TPOT alone is insufficient for this decision. They are lagging indicators, and the same latency symptom can arise from different bottlenecks. In particular, elevated TTFT may result either from prefill overload or from decode-side admission backpressure. In P/D-disaggregated serving such as SGLang [53], the decode worker reserves KV-cache capacity before prefill execution begins. Insufficient decode-side KV-cache headroom can block a request from entering prefill, even when prefill compute remains available. FluidToken therefore uses PPI and DPI to distinguish prefill deadline pressure from decode-side pressure before an SLO violation occurs.

Prefill Pressure Index (PPI). PPI predicts whether queued requests can complete prefill within their remaining TTFT budgets. Consider the first n queued requests in their current scheduling order, indexed by $i = 1, \dots, n$. For request i , let t_i^{arr} and L_i denote its arrival time and prompt length. Under

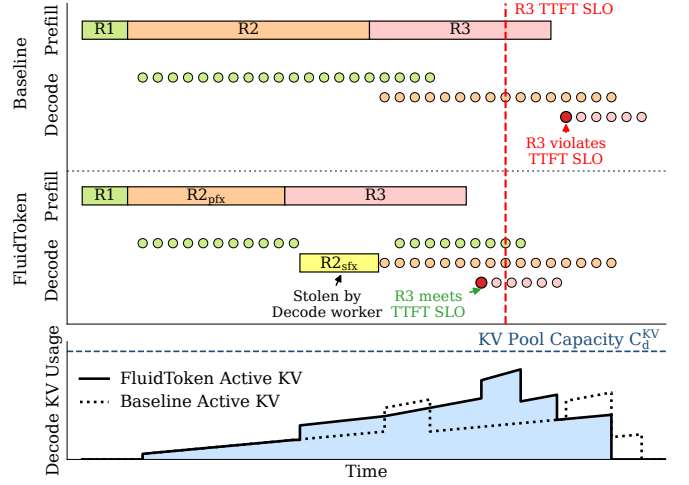


Fig. 7: FluidToken splits request $R2$ so $R3$ can begin prefill earlier and meet its TTFT SLO.

a TTFT SLO S_{TTFT} , its deadline is

$$t_i^{\text{ddl}} = t_i^{\text{arr}} + S_{\text{TTFT}} \quad (1)$$

An offline-calibrated predictor $\hat{T}_{\text{pre}}(L_i)$ estimates prefill latency. In our implementation, a linear model provides sufficient accuracy over the evaluated prompt-length range:

$$\hat{T}_{\text{pre}}(L_i) = c_1 L_i + c_0 \quad (2)$$

At the current time t^{now} , request i can meet its deadline only if the prefill worker can complete requests 1 through i within its deadline. Following performance guidance [2], we configure each prefill worker to process one request per batch: medium-to-long prefills already saturate GPU compute, so batching multiple prefill requests provides little additional throughput. As a result, queued prefills are served in order. We say request i is *TTFT-schedulable* if its predicted prefill completion time does not exceed its TTFT deadline:

$$\sum_{k=1}^i \hat{T}_{\text{pre}}(L_k) \leq t_i^{\text{ddl}} - t^{\text{now}} \quad (3)$$

We normalize this into a request-level pressure signal:

$$\text{PPI}_i = \frac{\sum_{k=1}^i \hat{T}_{\text{pre}}(L_k)}{\max(t_i^{\text{ddl}} - t^{\text{now}}, \epsilon)} \quad (4)$$

where $\epsilon > 0$ keeps the denominator positive when the deadline has expired. A value $\text{PPI}_i \leq 1$ indicates that request i is predicted to finish prefill within its remaining TTFT budget, whereas $\text{PPI}_i > 1$ indicates that i is at risk of missing its TTFT SLO. We summarize the pressure of the inspected queue window using

$$\text{PPI} = \max_{1 \leq i \leq n} \text{PPI}_i \quad (5)$$

Unlike observed TTFT, PPI is a leading signal: it detects deadline risk before violations occur and identifies which queued requests are at risk.

Decode Pressure Index (DPI). DPI determines whether the paired decode worker has sufficient execution and KV-cache headroom to accept a stolen suffix. For each decode worker d , let C_d^{KV} denote its total KV-cache capacity, C_d^{active} the capacity occupied by active decode requests, and C_d^{reserved} the capacity reserved for admitted requests awaiting KV transfer or decode execution. Its remaining allocable capacity is

$$C_d^{\text{alloc}} = C_d^{\text{KV}} - C_d^{\text{active}} - C_d^{\text{reserved}}. \quad (6)$$

Let N_d^{active} denote the current decode batch size, and let N_d^{TPOT} denote the largest decode batch size that satisfies the TPOT SLO. We decompose DPI into two components. The load-pressure component,

$$\text{DPI}_{d,\text{load}} = \max\left(\frac{C_d^{\text{active}}}{C_d^{\text{KV}}}, \frac{N_d^{\text{active}}}{N_d^{\text{TPOT}}}\right), \quad (7)$$

captures the load imposed by active decode requests. The admission-pressure component,

$$\text{DPI}_{d,\text{adm}} = 1 - \frac{C_d^{\text{alloc}}}{C_d^{\text{KV}}} \quad (8)$$

captures the fraction of KV-cache capacity already occupied or reserved and therefore unavailable for admitting new requests. FluidToken uses these signals to avoid stealing work to a decode worker that is already heavily loaded or lacks sufficient admission headroom.

3) *FluidToken Policy*: At each prefill scheduling iteration, FluidToken uses PPI to identify requests at risk of missing their TTFT SLOs and DPI to determine whether their paired decode workers can accept stolen work. It then adjusts the previously defined stealing ratios $\{\beta_i\}$ to restore TTFT schedulability while respecting decode-side limits.

Stealing-aware TTFT constraint. Token stealing shortens the work executed on the prefill worker, but request i 's offloaded suffix remains on its TTFT critical path. Let $L_{\text{pfx},i} = (1 - \beta_i)L_i$ and $L_{\text{sfx},i} = \beta_i L_i$ denote request i 's prefix and suffix lengths, and let $\hat{T}_{\text{pfx}}(L_{\text{pfx},i})$ and $\hat{T}_{\text{sfx}}(L_{\text{sfx},i})$ denote their predicted execution times on the prefill and decode workers, respectively. Since $t_i^{\text{ddl}} = t_i^{\text{arr}} + S_{\text{TTFT}}$, the suffix time reduces the deadline available to the prefill-side prefix:

$$t_i^{\text{pfx-ddl}} \triangleq t_i^{\text{ddl}} - \hat{T}_{\text{sfx}}(L_{\text{sfx},i}). \quad (9)$$

This deadline reserves the predicted suffix execution time but does not add a separate decode-side queueing term. Once the prefix KV transfer completes, an admitted suffix is scheduled at the next eligible iteration as a prefill forward pass rather than entering the decode waiting queue. Any delay imposed on the active decode batch is bounded separately by the decode-side stealing budget described below.

We evaluate a candidate stealing decision by applying the normal PPI schedulability condition in (3) to the modified stealing schedule: Under stealing, request i 's deadline becomes the prefix deadline $t_i^{\text{pfx-ddl}}$. The prefill worker still needs to process the queued requests ahead of request i , but each preceding request k contributes only its prefix work $\hat{T}_{\text{pfx}}(L_{\text{pfx},k})$. The stealing decision is TTFT-schedulable only if, for each

request, the accumulated prefix execution time of it and all requests ahead of it does not exceed its prefix deadline:

$$\sum_{k=1}^i \hat{T}_{\text{pfx}}(L_{\text{pfx},k}) \leq t_i^{\text{pfx-ddl}} - t^{\text{now}}, \quad \forall i. \quad (10)$$

For compactness, we express the stealing-aware TTFT constraint using the normalized form of PPI:

$$\text{PPI}_i^{\text{steal}} = \frac{\sum_{k=1}^i \hat{T}_{\text{pfx}}(L_{\text{pfx},k})}{\max(t_i^{\text{pfx-ddl}} - t^{\text{now}}, \epsilon)} \leq 1, \quad \forall i. \quad (11)$$

Decode-side stealing budget. Satisfying the prefill-side constraint is not sufficient: the stolen suffixes must also fit within the paired decode worker's TPOT and KV-cache headroom. For each decode worker d , FluidToken first requires

$$\text{DPI}_{d,\text{load}} < \text{DPI}_{\text{load}}^{\text{high}}, \quad (12)$$

where $\text{DPI}_{\text{load}}^{\text{high}}$ is the maximum active decode load under which token stealing is allowed.

For an eligible worker, FluidToken further bounds the amount of stolen work by its remaining TPOT, and the admission-pressure condition ensures sufficient KV-cache headroom for the stolen suffix. Let $\hat{T}_{\text{iter}}(N, C)$ denote the predicted latency of a regular decode iteration with a running batch size N and aggregate context length C . We model this latency using an offline-profiled linear predictor:

$$\hat{T}_{\text{iter}}(N, C) = a_C C + a_N N + a_0. \quad (13)$$

The worker's predicted TPOT slack is

$$\Delta_d^{\text{TPOT}} = S_{\text{TPOT}} - \hat{T}_{\text{iter}}(N_d^{\text{active}}, C_d^{\text{active}}). \quad (14)$$

A stolen suffix temporarily defers the decode iteration. Because TPOT is measured as the average inter-token latency over the output stream, a request may tolerate an occasional delayed decode iteration when earlier iterations have accumulated sufficient slack. FluidToken captures this stream-level slack by a multiplier η of the currently available slack:

$$\hat{T}_{\text{sfx}}(L_{\text{sfx},i}) \leq \eta \cdot \Delta_d^{\text{TPOT}}. \quad (15)$$

Setting $\eta = 1$ enforces a strict per-iteration bound, while $\eta > 1$ allows FluidToken to borrow slack accumulated across multiple decode iterations. Because the suffix-latency predictor is invertible, FluidToken can directly translate the available TPOT slack into a maximum admissible suffix length:

$$L_{d,\text{max}}^{\text{sfx},\text{TPOT}} = \hat{T}_{\text{sfx}}^{-1}(\eta \Delta_d^{\text{TPOT}}). \quad (16)$$

The per-prefill-worker stealing budget for decode worker d is:

$$B_d^{\text{steal}} = \begin{cases} 0, & \text{DPI}_{d,\text{load}} \geq \text{DPI}_{\text{load}}^{\text{high}} \\ \frac{L_{d,\text{max}}^{\text{sfx},\text{TPOT}}}{|P|}, & \text{otherwise.} \end{cases} \quad (17)$$

where $|P|$ is the number of prefill workers paired with d .

Request selection and online optimization. Given the advertised decode-worker budgets, FluidToken determines each queued request's stealing ratio β_i . The objective is to minimize

Algorithm 1 FluidToken Stealing-Ratio Selection Algorithm

Require: Prefill queue $\langle R_1, \dots, R_n \rangle$ in queue order; request length L_j for each request j ; stealing budget B_d^{steal} advertised by each decode worker d

Ensure: Stealing ratios $\{\beta_i\}_{i=1}^n$

```

1:  $stolen_j \leftarrow 0, \forall j$  ▷ tokens stolen from  $R_j$  so far
2:  $budget_d \leftarrow B_d^{\text{steal}}, \forall d$  ▷ remaining stealing budget for  $d$ 
3: for  $i \leftarrow 1$  to  $n$  do
4:   compute  $X_i$  using (19)
5:   if  $X_i \leq 0$  then
6:     continue ▷  $R_i$  already schedulable
7:   end if
8:    $need_i \leftarrow X_i$  ▷ remaining token deficit of  $R_i$ 
9:   for  $j \leftarrow 1$  to  $i - 1$  do ▷ earliest request first
10:    if  $need_i \leq 0$  then
11:      break
12:    end if
13:     $d \leftarrow d(j)$  ▷ decode worker paired with  $R_j$ 
14:     $take_{i,j} \leftarrow \min\{need_i, L_j - stolen_j, budget_d\}$ 
15:     $stolen_j \leftarrow stolen_j + take_{i,j}$ 
16:     $budget_d \leftarrow budget_d - take_{i,j}$ 
17:     $need_i \leftarrow need_i - take_{i,j}$ 
18:  end for
19: end for
20:  $\beta_j \leftarrow stolen_j / L_j, \forall j$ 
21: return  $\{\beta_i\}_{i=1}^n$ 

```

the number of requests predicted to violate TTFT SLO, as indicated by $\text{PPI}_i^{\text{steal}} > 1$. Let $\pi_{i,d} \in \{0, 1\}$ indicate whether request i is paired with decode worker d . FluidToken solves the following optimization problem:

$$\begin{aligned}
\min_{\{\beta_i\}} \quad & \sum_{i=1}^n \mathbf{1}[\text{PPI}_i^{\text{steal}} > 1] \\
\text{s.t.} \quad & \sum_{i: \pi_{i,d}=1} \beta_i L_i \leq B_d^{\text{steal}}, \quad \forall d, \\
& 0 \leq \beta_i \leq 1, \quad \forall i.
\end{aligned} \tag{18}$$

The first constraint limits the total stolen suffix tokens assigned to each paired decode worker and the second constraint bounds the stealing ratios to their valid range. Each prefill worker solves the optimization locally using the per-prefill budget advertised by each paired decode worker.

To solve this optimization, we make a key observation: the same stealing budget provides equal or more benefits when used to steal work from requests earlier in the queue. Consider two requests i and j with j ahead of i in the queue, that are paired with the same decode worker. Under the linear prefill-latency predictor in (2), stealing the same number of tokens from either request removes the same amount of prefill execution time. However, stealing from j advances every request that would benefit from stealing from i , as well as the requests between them. Allocating the decode worker's stealing budget to j achieves at least the same reduction in predicted TTFT violations as allocating it to i . FluidToken consequently allocates each decode worker's stealing budget to the earliest eligible requests first.

Under the linear prefill-latency predictor in (2), each stolen token reduces the predicted prefill workload by c_1 . We therefore define the token deficit of request i directly as:

$$X_i \triangleq \frac{1}{c_1} \left(\sum_{k \leq i} \hat{T}_{\text{pfx}}(L_{\text{pfx},k}) - (t_i^{\text{pfx-ddl}} - t^{\text{now}}) \right), \tag{19}$$

This equation converts i 's predicted TTFT lateness into the number of tokens that must be removed from the prefill queue ahead of it to make it TTFT-schedulable.

A value $X_i \leq 0$ means i is predicted to meet its deadline. To close i 's token deficit, the algorithm considers preceding requests $j < i$ in queue order.

A request is *eligible* to supply stolen work if it has unstolen prompt tokens and its paired decode worker has remaining stealing budget.

FluidToken solves (18) using the greedy procedure in Algorithm 1. For each request predicted to miss its deadline, the algorithm draws tokens from the earliest eligible preceding requests until either its deficit X_i is closed or no eligible request remains. The amount taken from each request j is bounded by the remaining deficit, request j 's unstolen prompt tokens, and the remaining stealing budget of its paired decode worker.

C. FluidRole: In-Place Role Switching

FluidToken handles transient prefill pressure without changing worker roles. When phase imbalance persists, FluidRole changes the effective P/D worker ratio by switching an existing worker between prefill and decode roles in place.

1) Role-State Expansion and Bidirectional Live-Role Migration: FluidRole extends the runtime with a role-switching state machine (Fig. 8) comprised of three states: ACTIVE, DRAIN, and SWITCH. In ACTIVE, the worker executes the scheduler and KV-transfer path of its current role. Upon receiving a switch request, the worker transitions through three states: $\text{ACTIVE}(\text{old role}) \rightarrow \text{DRAIN} \rightarrow \text{SWITCH} \rightarrow \text{ACTIVE}(\text{new role})$. It first enters DRAIN, where it stops accepting new work and waits for in-flight operations to reach a safe boundary. It then enters SWITCH, where FluidRole activates the target role's queues, transfer handlers, and scheduler state.

FluidRole changes a worker's serving role while preserving its model placement and parallelism configuration. The worker therefore reuses its existing model-parallel communication groups throughout the transition. When switching into the decode role, FluidRole initializes the required decode CUDA graphs within the existing worker process, without restarting the engine or reloading model weights. In the current implementation, these graphs are captured on the first switch into decode and remain cached for subsequent switches, so only the first transition incurs the capture overhead. Capturing the graphs for both roles at startup would eliminate this first-switch cost, at the expense of additional GPU memory for retaining both sets of CUDA graph state. The worker finally returns to ACTIVE and resumes serving under the new role.

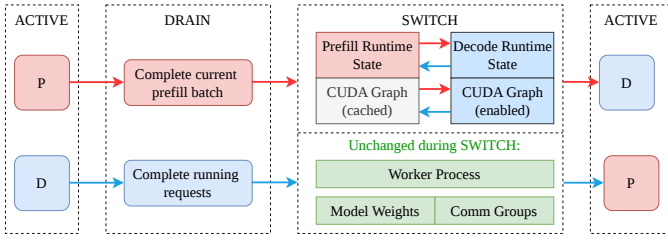


Fig. 8: FluidRole role-switching state machine.

FluidRole switches roles without dropping requests or causing downtime, even when the selected worker is serving requests under its current role. Unlike restart-based reconfiguration, role switching avoids the downtime caused by process startup, model loading, and distributed initialization, but still requires the worker to drain role-specific in-flight work before the transition can complete. The key idea is to reassign work that has not yet started while draining work that is already in progress. Queued requests can be removed from the worker and reassigned to another worker with the same current role. In-progress requests remain on the selected worker until they reach a safe transition boundary. Because the amount and duration of in-progress work differ between prefill and decode, the resulting drain cost is directional.

For a $P \rightarrow D$ switch, the worker stops admitting new prefill requests, reassigns queued requests that have not started execution, completes its current prefill batch, and flushes any outstanding KV transfers. Its drain time is therefore bounded primarily by the remaining prefill execution and non-overlapped KV-transfer time. For a $D \rightarrow P$ switch, the worker stops admitting new decode requests and allows its active requests to finish before activating the prefill role. These requests already hold local KV state and may continue generating until EOS or the configured maximum generation length. Because their remaining output lengths are unknown at switch time, the drain time depends on the longest remaining active sequence and can be substantially larger than for a $P \rightarrow D$ switch. We quantify these overheads in § V-H.

2) *FluidRole Policy*: FluidRole aggregates the PPI and DPI signals over a longer observation window to distinguish sustained phase imbalance from the transient pressure handled by FluidToken. For the prefill pool, FluidRole tracks

$$\overline{\text{PPI}} = \max_{p \in \mathcal{P}} \text{PPI}^{95}(p), \quad (20)$$

where $\text{PPI}^{95}(p)$ is the 95th percentile, over the observation window, of prefill worker p 's queue-level pressure ($\max_i \text{PPI}_i$). This allows FluidRole to detect localized overload that may cause TTFT violations even when the pool-wide average remains moderate. For the decode pool, FluidRole tracks the average sustained decode pressure:

$$\overline{\text{DPI}}_{\text{load}} = \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} \text{DPI}_{d,\text{load}}, \quad \overline{\text{DPI}}_{\text{adm}} = \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} \text{DPI}_{d,\text{adm}}. \quad (21)$$

The load component captures pressure from active decode requests relative to the TPOT-safe concurrency limit, whereas the admission component captures committed KV-cache occupancy from active and reserved requests.

Given the persistent aggregate pressure signals, FluidRole first identifies which phase would benefit from additional capacity and then checks whether the opposite phase can safely relinquish one worker. A $D \rightarrow P$ switch is considered when prefill pressure remains high and the decode pool has sufficient active-load headroom:

$$\overline{\text{PPI}} > \text{PPI}^{\text{high}} \quad \wedge \quad \overline{\text{DPI}}_{\text{load}} < \text{DPI}_{\text{load}}^{\text{high}}. \quad (22)$$

Conversely, a $P \rightarrow D$ switch is considered when decode-side admission pressure remains high and the prefill pool has sufficient headroom:

$$\overline{\text{DPI}}_{\text{adm}} > \text{DPI}_{\text{adm}}^{\text{high}} \quad \wedge \quad \overline{\text{PPI}} < \text{PPI}^{\text{low}}. \quad (23)$$

Admission pressure may become critical before active decode load reaches its limit. KV capacity can already be committed to admitted requests that are still waiting for KV transfer or decode execution. Consequently, DPI_{adm} captures pending decode demand that may not yet appear in DPI_{load} . If both phases exhibit persistent pressure, FluidRole does not move capacity between them, since relieving one bottleneck would worsen the other. This condition indicates insufficient aggregate serving capacity and could instead be handled through cluster-level admission control or autoscaling.

FluidRole commits to a role switch only when the same switch recommendation persists for N_{persist} consecutive intervals. This prevents short-lived pressure fluctuations from triggering role changes. Once this happens, FluidRole selects the worker with the smallest estimated drain cost. For $D \rightarrow P$ transitions, where the remaining output lengths are unknown, FluidRole uses the number of active decode requests as a lightweight proxy for drain time. The controller switches one worker at a time and favors the candidate expected to reach the safe switching boundary with the least disruption. After a switch, the controller enforces a cooldown period T_{cooldown} before considering another transition, allowing the new P/D configuration to take effect. Together, N_{persist} and T_{cooldown} prevent rapid or repeated role changes when workload demand fluctuates near the decision thresholds.

V. EVALUATION

A. Evaluation Methodology

Implementation: We implement FluidPD on top of SGLang [53], a production-grade LLM serving engine with P/D disaggregation support. FluidPD extends SGLang's disaggregated runtime (v0.5.4) with FluidToken and FluidRole while preserving SGLang's standard P/D execution flow, including prefill-side scheduling, decode-side KV-cache preallocation, asynchronous KV transfer, and CUDA Graph execution. A deployment with N prefill workers and M decode workers is denoted as $NPMD$. Our current predictor implementation relies on offline profiling. Online recalibration from observed

execution times is a natural extension for tracking runtime variation and workload drift.

Models and workloads: We evaluate FluidPD on three LLMs with different serving characteristics: Llama-3.1-8B, Qwen3-14B, and Qwen3-30B-A3B (MoE). We evaluate FluidPD using workloads derived from the Azure LLM Inference trace [42], which provides separate Azure-code and Azure-conversation traces spanning one week each. These traces contain substantial variation in request mix, arrival rate, and short-timescale arrival burstiness. Because replaying the complete 168-hour trace for every configuration is prohibitively expensive, we construct two representative workloads. *Azure (staged)* contains 30K requests arranged as three 10K-request stages: Azure-conversation, Azure-code, and Azure-conversation.

Azure-code is more prefill-heavy because its requests tend to have longer prompts and shorter generations, whereas Azure-conversation places relatively more pressure on decode through longer output streams. The staged workload therefore captures sustained shifts in the preferred P:D ratio. *Azure (mixed)* contains 10K naturally interleaved requests from a bursty trace window, capturing the short-timescale variation described in § III-B. We replay both workloads at controlled request rates.

Baselines and configurations: We compare FluidPD against SGLang v0.5.4, vLLM v0.26.0, and Dynamo v1.3.0. The SGLang and vLLM baselines use static 4P4D deployments, while FluidPD starts from the same 4P4D allocation and adapts its effective P:D capacity in place. We evaluate Dynamo both as a static 4P4D deployment under the same 8-GPU budget and, in a separate autoscaling comparison, as a 6-GPU deployment that can grow to 8 GPUs. All evaluated configurations use round-robin request routing. We do not directly compare with TaiChi, TokenScale, or Libra [39] (DynaServe in its preprint version) because no public implementations were available at the time of evaluation.

Metrics: We use overall SLO attainment as the primary metric, following [35], [48], [54]. TTFT and TPOT SLOs define upper bounds for the corresponding latency metrics, and attainment rate is the percentage of requests satisfying each bound. Overall SLO attainment is the percentage of requests that satisfy both bounds. In the main results, TPOT SLO is 100 ms for all models, while TTFT SLO is 1000 ms for Qwen3-30B-A3B, 2000 ms for Llama-3.1-8B, and 3000 ms for Qwen3-14B.

Hardware testbed: All experiments run on an AWS server with $8 \times$ NVIDIA A100-80GB GPUs interconnected via NVLink, using one GPU per serving instance.

B. Main Results

Fig. 9a evaluates whether in-place P/D elasticity improves SLO attainment over static 8-GPU deployments as the offered request rate increases. Across both Azure workloads, FluidPD improves overall SLO attainment under pressure. Compared with static SGLang, FluidPD improves overall SLO attainment by up to 94.6 percentage points on Azure (staged) and by up to 74.3 percentage points on Azure (mixed). The largest

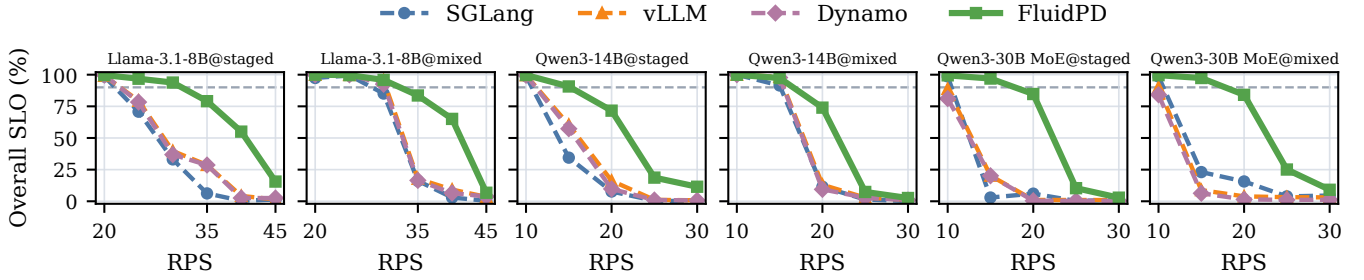
gains appear at low and moderate offered loads, where the fixed baselines become mismatched to workload phase demand even though the deployment still has usable aggregate capacity. FluidPD can exploit this headroom by reassigning workers for sustained shifts and temporarily borrowing decode-side slack for shorter bursts. At higher offered loads, the deployment approaches saturation, leaving less spare capacity for either mechanism.

Fig. 9b breaks down this effect for Llama-3.1-8B on Azure (mixed). In SGLang’s P/D-disaggregated execution, both sustained phase shifts and transient bursts first create admission-side pressure at the prefill workers, increasing TTFT violation. As a result, FluidPD’s overall SLO improvement primarily comes from higher TTFT attainment, while TPOT attainment remains close to the baselines through most of the sweep. At the highest offered load, FluidPD’s TPOT attainment decreases slightly. This happens because partial-prefill execution on decode workers consumes decode-side slack; once the system is saturated, stolen prefill work can interfere with decode iterations enough to create TPOT violations.

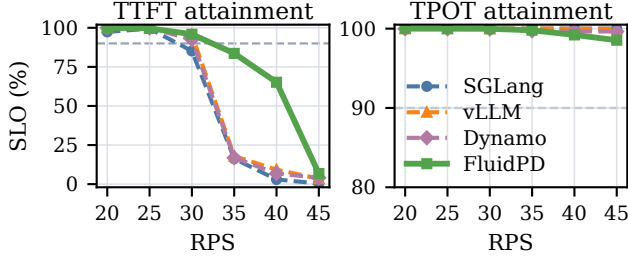
We next compare FluidPD with autoscaling to determine whether adding workers can address the same P/D imbalance that FluidPD resolves within the running deployment. We run Llama-3.1-8B on Azure (mixed) and Azure (staged). Both systems start from the same 6-GPU 3P3D configuration. FluidPD remains within these six GPUs, whereas Dynamo may scale the deployment to eight GPUs. Fig. 9c shows that autoscaling reacts slowly to the imbalance timescales in both workloads. On Azure (mixed), the dominant event is a short prefill-heavy burst. At 20 RPS, FluidPD achieves 97.6% overall SLO attainment, compared with 9.3% for Dynamo. Dynamo initiates scale-out after the burst begins, but the new worker becomes available only hundreds of seconds later, after the burst has already caused substantial TTFT violations. In contrast, FluidPD responds within the existing deployment: FluidToken borrows available decode-side slack to execute bounded prefill work and reduce queueing without waiting for additional workers. Azure (staged) contains a longer-lasting demand shift and is therefore more favorable to autoscaling. Nevertheless, at 20 RPS, FluidPD achieves 92.6% overall SLO attainment, whereas Dynamo achieves 5.2%. Under Dynamo’s active planner, the deployment repeatedly changes its P/D allocation, including removing workers and later adding them back, and does not converge quickly to a configuration that matches the workload. FluidPD instead adjusts the P/D split among already-running workers through FluidRole and uses FluidToken to absorb shorter pressure intervals.

C. Why FluidPD Helps: Time-Series Breakdown

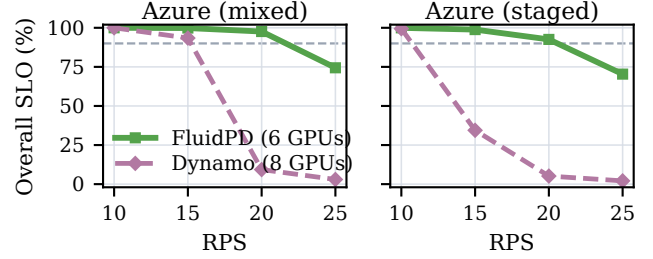
We examine the time-series behavior of FluidPD on Azure (staged). The initial 4P4D allocation is a good fit for Azure-conversation and keeps prefill queueing low. When the workload shifts to Azure-code, input-token demand increases and the same allocation becomes prefill-constrained. FluidPD responds to this sustained imbalance through FluidRole, which performs the observed 4P4D $\rightarrow$ 5P3D transition. The ad-



(a) Overall SLO attainment on Azure workloads on static 4P4D allocation. FluidPD consistently outperforms all baselines.



(b) TTFT and TPOT contribution on Llama-3.1-8B for Azure (mixed).



(c) FluidPD-static versus Dynamo-autoscaling on Azure workloads.

Fig. 9: Main results on Azure workloads across Llama-3.1-8B, Qwen3-14B, and Qwen3-30B-A3B MoE.

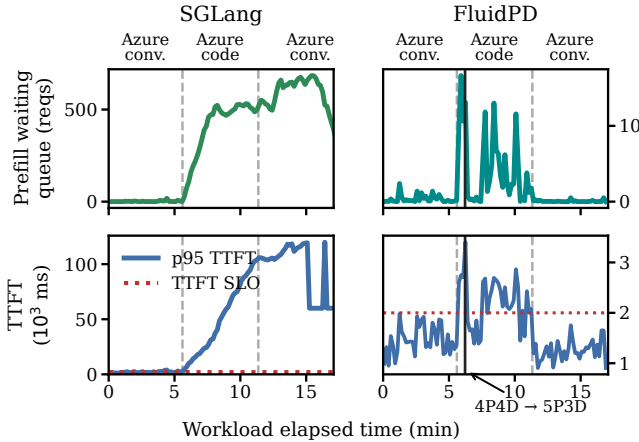


Fig. 10: Time-series comparison on Azure (staged).

ditional prefill worker bounds queue growth, allowing p95 TTFT to recover below the SLO after the workload returns to Azure-conversation. In contrast, SGLang cannot adjust its P/D allocation: its prefill queue grows throughout the Azure-code stage and remains elevated through the end of the trace. During shorter pressure intervals that do not justify a role switch, FluidToken can instead borrow decode-side slack for bounded prefill work without changing the worker allocation. SGLang cancels requests that wait more than 60s for their first token. These requests therefore leave the queue as failures, producing the saw-tooth pattern in the static TTFT timeline rather than a continuously increasing curve.

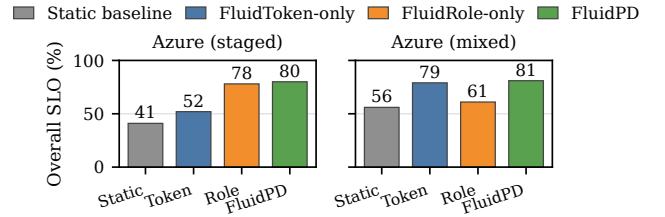


Fig. 11: Component ablation on Llama-3.1-8B.

D. Component Ablation

To isolate the contribution of each mechanism, we compare FluidPD against SGLang (static), FluidToken-only, and FluidRole-only on Llama-3.1-8B. Both workloads use a 4P4D starting allocation. Azure (mixed) runs at 35 RPS and Azure (staged) runs at 30 RPS. Fig. 11 shows that the mechanisms contribute differently across workloads. On Azure (staged), FluidRole provides the larger individual benefit because it adapts the worker allocation to sustained shifts in P:D demand. On Azure (mixed), FluidToken provides the larger benefit because it can respond directly to transient prefill pressure. Full FluidPD performs best across both workloads by combining these complementary timescales of adaptation.

E. Sensitivity to SLO Targets

We evaluate Llama-3.1-8B on Azure (mixed) across the RPS sweep for three latency profiles: strict (1000 ms TTFT / 50 ms TPOT), medium (2000 ms TTFT / 100 ms TPOT), and relaxed (3000 ms TTFT / 150 ms TPOT). We focus on TTFT because TPOT attainment remains close to 100% at these

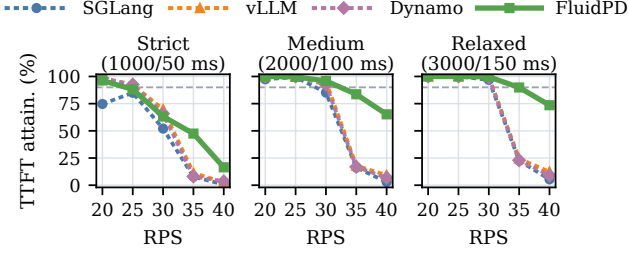


Fig. 12: Sensitivity to SLO targets on Azure (mixed). We omit TPOT because it remains close to 100%.

operating points. Across all SLO profiles, FluidPD achieves TTFT attainment comparable to or better than the baselines. As the SLO constraints relax, the available decode-side slack grows, allowing FluidToken to offload more prefill work and widen FluidPD’s advantage.

F. Hyperparameter Sensitivity

FluidPD exposes several control parameters that determine how aggressively it reuses capacity across phases. For FluidToken, one important parameter is the budget factor η , which scales the TPOT-derived suffix-execution bound. A larger η relaxes this bound, allowing the solver to admit larger stolen suffixes to reduce prefill queuing but introduces decode interference. In our runs, we find $\eta = 1.25$ to balance this trade-off. For FluidRole, the key parameters are the PPI and DPI thresholds that control when the controller changes the P:D allocation. These thresholds are important because they directly control reaction time: overly conservative values delay adaptation, while overly aggressive values can cause churn. We find PPI-low = 0.6, PPI-high = 1.0, DPI-load = 0.6, and DPI-admission = 0.2 to be a good combination.

G. Predictor Model Accuracy

FluidPD uses lightweight latency predictors to estimate prefill execution, stolen-suffix execution, and decode-iteration latency. We calibrate these predictors offline for each model and hardware configuration evaluated in this work. The controller does not require a universally valid analytical model. It only requires sufficiently accurate estimates over the operating range of the deployed configuration. Fig. 13 compares predicted and measured execution times over the profiled range. The predictions closely track the measurements, with relative error within 2–10% for most profiled points. This accuracy is sufficient for online control because FluidPD uses the predictions to enforce conservative stealing and role-switching constraints, rather than to estimate exact end-to-end request latency.

H. Control and Switching Overheads

We also measure the runtime overhead of FluidPD’s online control logic. Computing PPI, DPI, and suffix-stealing budgets uses scheduler metadata already maintained by SGLang, including queue state, request lengths, decode batch size, context

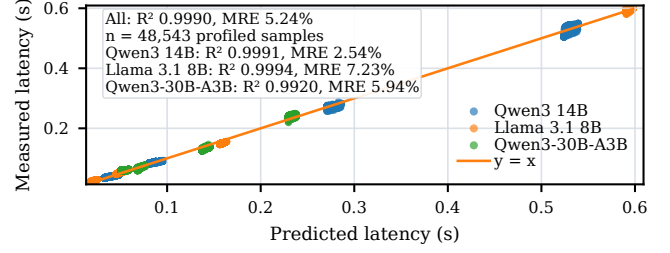


Fig. 13: Accuracy of the latency models used by FluidPD. MRE denotes mean relative error.

length, and KV-cache occupancy. The resulting CPU-side overhead is less than 0.50 ms per control iteration. SGLang’s overlap scheduler pipelines CPU-side scheduling with GPU execution, so this overhead is largely hidden from the prefill and decode critical paths. FluidToken introduces additional GPU work only when a stolen suffix executes on a decode worker. This cost is included in the end-to-end latency and SLO results. FluidRole additionally incurs the time required for the selected worker to reach a safe transition boundary. A P→D transition takes approximately 1 s, dominated by completing the current prefill and flushing outstanding KV transfers. A D→P transition takes up to 12 s, depending on the active decode requests that must finish before the worker can change roles. These transitions preserve admitted requests and avoid process restart and model reloading.

VI. CONCLUSION

In this work, we present FluidPD, a P/D-disaggregated LLM serving system that adapts capacity in place under both transient bursts and sustained shifts in phase demand. FluidToken uses SLO-aware token-level work stealing to exploit decode-side slack, while FluidRole rebalances the running P/D worker split through in-place role switching. Guided by predictive pressure signals, FluidPD improves overall SLO attainment by up to 94.6 percentage points on Azure trace workloads while largely preserving TPOT attainment outside overloaded ranges. These results show that P/D imbalance can often be addressed using capacity already present in the deployment, before resorting to additional GPUs.

ACKNOWLEDGMENTS

This research was supported by the National Science Foundation (NSF) under Grant No. 2441601. The work utilized the Delta and DeltaAI system at the National Center for Supercomputing Applications (NCSA) and Jetstream2 at Indiana University through allocation CIS240055 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296. The Delta advanced computing resource is a collaborative effort between the University of Illinois Urbana-Champaign and NCSA, supported by the NSF (award OAC 2005572) and the State of Illinois. UIUC SSAIL

Lab is supported by research funding and gift from IBM, Google, Amazon, and AMD.

REFERENCES

- [1] “AIBrix: Towards scalable, cost-effective large language model inference infrastructure,” *arXiv preprint arXiv:2504.03648*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.03648>
- [2] “NVIDIA Dynamo: Disaggregation and performance tuning,” <https://docs.nvidia.com/dynamo/v0.8.0/user-guides/tuning-disaggregated-performance.html>, 2026, accessed July 27, 2026.
- [3] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, “Sarathi-Serve: Taming throughput-latency tradeoff in LLM inference with chunked prefills,” 2024.
- [4] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He, “Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale,” in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–15.
- [5] Anthropic, “Claude Code,” <https://claude.com/product/claude-code>, 2026, accessed: 2026-07-31.
- [6] S. Arun, A. Parayil, S. Bharadwaj, R. St. Amant, and V. Rühle, “Towards load-aware prefill deflection for disaggregated LLM serving,” *arXiv preprint arXiv:2607.02043*, 2026.
- [7] A. Bari, P. Hegde, and G. de Veciana, “Optimal scheduling algorithms for LLM inference: Theory and practice,” *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 9, no. 3, 2025.
- [8] Y. Chen, W. Cui, H. Zhao, Z. Xu, X. Fan, X. Chen, Y. Zhou, S. Sun, B. He, and Q. Chen, “Towards high-goodput LLM serving with prefill-decode multiplexing,” in *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2026.
- [9] N. Corporation, “Dynamo inference framework — nvidia developer,” 3 2026, [Online]. [Online]. Available: <https://developer.nvidia.com/dynamo>
- [10] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 16344–16359.
- [11] J. Fang, Y. Yu, C. Zhao, and J. Zhou, “Turbotransformers: an efficient gpu serving system for transformer models,” in *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2021, pp. 389–402.
- [12] J. Feng, Y. Huang, R. Zhang, S. Liang, M. Yan, and J. Wu, “WindServe: Efficient phase-disaggregated LLM serving with stream-based dynamic scheduling,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA 25)*, 2025, pp. 1283–1295.
- [13] S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao, “Model tells you what to discard: Adaptive KV cache compression for LLMs,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [14] GitHub, “Github copilot,” <https://github.com/features/copilot>, 2026, accessed: 2026-04-15.
- [15] K. Hong, X. Li, L. Chen, Q. Mao, G. Dai, X. Ning, S. Yan, Y. Liang, and Y. Wang, “SOLA: Optimizing SLO attainment for large language model serving with state-aware scheduling,” in *Proceedings of Machine Learning and Systems*, 2025.
- [16] C. Hu, H. Huang, L. Xu, X. Chen, J. Xu, S. Chen, H. Feng, C. Wang, S. Wang, Y. Bao, N. Sun, and Y. Shan, “Inference without interference: Disaggregate LLM inference for mixed downstream workloads,” *arXiv preprint arXiv:2401.11181*, 2024.
- [17] Y. Jiang, F. Fu, X. Yao, T. Wang, B. Cui, A. Klimovic, and E. Yoneki, “ThunderServe: High-performance and cost-efficient LLM serving in cloud environments,” in *Proceedings of Machine Learning and Systems*, 2025.
- [18] Y. Jin, T. Wang, H. Lin, M. Song, P. Li, Y. Ma, Y. Shan, Z. Yuan, C. Li, Y. Sun, T. Wu, X. Chu, R. Huan, L. Ma, X. You, W. Zhou, Y. Ye, W. Liu, X. Xu, Y. Zhang, T. Dong, J. Zhu, Z. Wang, X. Ju, J. Song, H. Cheng, X. Li, J. Ding, H. Guo, and Z. Zhang, “P/D-Serve: Serving disaggregated large language model at scale,” *arXiv preprint arXiv:2408.08147*, 2024.
- [19] A. K. Kamath, R. Prabhu, J. Mohan, S. Peter, R. Ramjee, and A. Panwar, “POD-Attention: Unlocking full prefill-decode overlap for faster LLM inference,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 897–912.
- [20] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with PagedAttention,” in *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [21] R. Lai, H. Liu, C. Lu, Z. Liu, S. Cao, S. Shao, Y. Zhang, L. Mai, and D. Ustiugov, “TokenScale: Timely and accurate autoscaling for disaggregated LLM serving with token velocity,” *arXiv preprint arXiv:2512.03416*, 2025.
- [22] R. Li, R. Du, Z. Chu, S. Zhao, C. Han, Z. Shi, Y. Shao, H. Han, L. Huang, Z. Liu, and S. Liu, “Taming the chaos: Coordinated autoscaling for heterogeneous and disaggregated LLM inference,” *arXiv preprint arXiv:2508.19559*, 2025.
- [23] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, “SnapKV: LLM knows what you are looking for before generation,” in *Advances in Neural Information Processing Systems*, 2024.
- [24] Z. Li, L. Zheng, Y. Zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez, and I. Stoica, “{AlpaServe}: Statistical multiplexing with model parallelism for deep learning serving,” in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023, pp. 663–679.
- [25] Z. Li, J. Liu, Z. Xu, Y. Zhang, T. Rabbani, and C. Zhang, “Not all prefills are equal: PPD disaggregation for multi-turn LLM serving,” in *Proceedings of the 43rd International Conference on Machine Learning*, 2026.
- [26] J. Liao, M. Xu, W. Zheng, Y. Wang, K. Ye, R. Buyya, and C. Xu, “DOPD: A dynamic PD-disaggregation architecture for maximizing goodput in LLM inference serving,” *arXiv preprint arXiv:2511.20982*, 2025.
- [27] llm-d contributors, “llm-d: Kubernetes-native distributed LLM inference,” <https://github.com/llm-d/llm-d>, 2025, accessed: 2026-04-06.
- [28] X. Miao, G. Oliaro, Z. Zhang, X. Cheng, Z. Wang, Z. Zhang, R. Y. Y. Wong, A. Zhu, L. Yang, X. Shi, C. Shi, Z. Chen, D. Arfeen, R. Abhyankar, and Z. Jia, “Specinfer: Accelerating large language model serving with tree-based speculative inference and verification,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 932–949.
- [29] T. Mitra, R. Borkar, N. Bhatia, R. Matas, S. Raj, D. Mudigere, R. Zhao, M. Golub, A. Dutta, S. Madduri, D. Jani, B. Pharris, and B. Darvish Rouhani, “Beyond the buzz: A pragmatic take on inference disaggregation,” *arXiv preprint arXiv:2506.05508*, 2025.
- [30] Nous Research, “Hermes agent,” <https://github.com/NousResearch/hermes-agent>, 2026, accessed: 2026-07-31.
- [31] NVIDIA, “Nvidia/fastertransformer: Transformer related optimization, including bert, gpt,” 2025, [accessed 2025-11-17]. [Online]. Available: <https://github.com/NVIDIA/FasterTransformer>
- [32] OpenAI, “ChatGPT overview,” <https://chatgpt.com/overview>, 2026, accessed: 2026-04-15.
- [33] OpenAI, “Codex in ChatGPT,” <https://openai.com/codex/>, 2026, accessed: 2026-07-31.
- [34] OpenClaw contributors, “OpenClaw: Personal AI assistant,” <https://github.com/openclaw/openclaw>, 2026, accessed: 2026-07-31.
- [35] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative llm inference using phase splitting,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 118–132.
- [36] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, “Efficiently scaling transformer inference,” *Proceedings of machine learning and systems*, vol. 5, pp. 606–624, 2023.
- [37] R. Prabhu, A. Nayak, J. Mohan, R. Ramjee, and A. Panwar, “vattention: Dynamic memory management for serving llms without pagedattention,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 1133–1150.
- [38] R. Qin, Z. Li, W. He, M. Zhang, Y. Wu, W. Zheng, and X. Xu, “Mooncake: A KVCache-centric disaggregated architecture for LLM serving,” *arXiv preprint arXiv:2407.00079*, 2024.
- [39] C. Ruan, Y. Chen, D. Tian, Y. Shi, Y. Wu, J. Li, and C. Li, “Libra: Flexible request partitioning and scheduling for serving unbalanced and dynamic LLM workloads,” in *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, 2026, pp. 1243–1258.

- [40] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “Flexgen: High-throughput generative inference of large language models with a single gpu,” in *International Conference on Machine Learning*. PMLR, 2023, pp. 31 094–31 116.
- [41] M. Stern, N. Shazeer, and J. Uszkoreit, “Blockwise parallel decoding for deep autoregressive models,” *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [42] J. Stojkovic, C. Zhang, Í. Goiri, J. Torrellas, and E. Choukse, “Dynamollm: Designing llm inference clusters for performance and energy efficiency,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025, pp. 1348–1362.
- [43] B. Sun, Z. Huang, H. Zhao, W. Xiao, X. Zhang, Y. Li, and W. Lin, “Llumnix: Dynamic scheduling for large language model serving,” in *18th USENIX symposium on operating systems design and implementation (OSDI 24)*, 2024, pp. 173–191.
- [44] Y. Tang, T. Lan, X. Huang, H. Lu, and W. Chen, “SCORPIO: Serving the right requests at the right time for heterogeneous SLOs in LLM inference,” *arXiv preprint arXiv:2505.23022*, 2025.
- [45] C. Wang, P. Zuo, Z. Chen, Y. Liang, Z. Yu, and M.-C. Yang, “TaiChi: Prefill-decode aggregation or disaggregation? unifying both for goodput-optimized LLM serving,” *arXiv preprint arXiv:2508.01989*, 2025.
- [46] X. Wang, Y. Xiong, Y. Wei, M. Wang, and L. Li, “Lightseq: A high performance inference library for transformers,” in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Industry Papers*, 2021, pp. 113–120.
- [47] Y. Wang, Y. Chen, Z. Li, X. Kang, Y. Fang, Y. Zhou, Y. Zheng, Z. Tang, X. He, R. Guo, X. Wang, Q. Wang, A. C. Zhou, and X. Chu, “Burstgpt: A real-world workload dataset to optimize llm serving systems,” in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, 2025, pp. 5831–5841.
- [48] Y. Wu, T. Liu, Y. Zeng, S. Wu, J. Xiong, X. Dong, H. Yang, K. Zhang, and J. Li, “Arrow: Adaptive scheduling mechanisms for disaggregated LLM inference architecture,” *arXiv preprint arXiv:2505.11916*, 2025.
- [49] Y. Xu, Z. Mao, X. Mo, S. Liu, and I. Stoica, “Pie: Pooling cpu memory for llm inference,” *arXiv preprint arXiv:2411.09317*, 2024.
- [50] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *Proceedings of the 11th International Conference on Learning Representations (ICLR 2023)*, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>
- [51] Z. Ye, L. Chen, R. Lai, W. Lin, Y. Zhang, S. Wang, T. Chen, B. Kasikci, V. Grover, A. Krishnamurthy, and L. Ceze, “FlashInfer: Efficient and customizable attention engine for LLM inference serving,” *arXiv preprint arXiv:2501.01005*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.01005>
- [52] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A distributed serving system for Transformer-Based generative models,” in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.
- [53] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez, C. Barrett, and Y. Sheng, “SGLang: Efficient execution of structured language model programs,” *arXiv preprint arXiv:2312.07104*, 2023.
- [54] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “{DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 193–210.